

Prompts & Design Documentation

Unified Enterprise AI Agent — Track 3

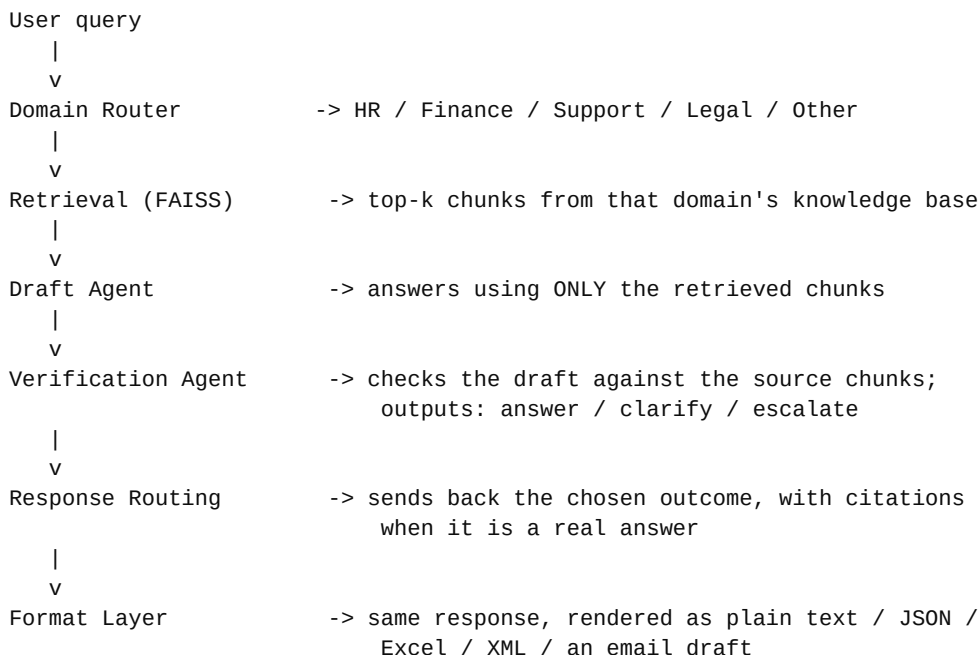
KOHLER-MITWPU AI Research Centre — Case Study Submission

Prepared by Raghav Uday Menon | B.Tech CSE (AI & DS), MIT-WPU

This document contains the exact prompts running in the deployed system, the reasoning behind the key design decisions in the pipeline, a short account of how the project was actually built and debugged, and a summary of the evaluation results. It is meant to be read alongside the GitHub repository and the live demo, not instead of them.

1. System Overview

The agent answers employee and customer questions across four domains — HR, Finance, Customer Support, and Legal/Compliance — by routing each query to a domain-specific knowledge base, retrieving the most relevant source chunks, drafting an answer strictly from that retrieved context, and then independently verifying the draft before it is shown to the user. Verification is what the rest of this document mostly explains: instead of always returning an answer with the same confident tone, the system decides between three outcomes — answer, ask a clarifying question, or escalate to a human team — based on whether the draft is actually grounded in the retrieved documents.



All three LLM calls (classify, draft, verify) currently run on Google's Gemini **gemini-3.5-flash-lite** model through the `google-genai` SDK. Embeddings for retrieval are computed locally with **sentence-transformers** (`all-MiniLM-L6-v2`) so no API call is needed for that step, and search runs over four separate **FAISS** indexes, one per domain.

2. System Prompts (Verbatim, as Deployed)

The three prompts below are copied directly from `src/agents.py` in the submitted repository — not an earlier draft. Each one was iterated on after being caught failing on a specific real test case; Section 3 explains what those failures were and how the fix is reflected in the wording below.

CLASSIFY_DOMAIN_SYSTEM_PROMPT

You are a routing classifier for an enterprise support assistant.

Classify the user's query into exactly one of the following domains:

- HR: employee benefits, leave, onboarding, performance reviews, workplace conduct, compensation
- FINANCE: expenses, budgets, invoices, procurement, payments, reimbursement, tax
- SUPPORT: product setup, troubleshooting, warranty, returns, device pairing
- LEGAL: contracts, compliance, data privacy, intellectual property, regulatory matters
- OTHER: anything that does not clearly fit HR, FINANCE, SUPPORT, or LEGAL

Respond with exactly one word: HR, FINANCE, SUPPORT, LEGAL, or OTHER.

Do not include punctuation, explanation, or any other text.

DRAFT_ANSWER_SYSTEM_PROMPT

You are an enterprise knowledge assistant. Answer the user's question using ONLY the information contained in the provided context chunks. Do not use outside knowledge and do not guess.

The conversation history may contain earlier turns in this exchange - use it to resolve follow-up questions (e.g. pronouns, "what about...", implied subjects) so your answer addresses what the user actually means.

If the answer depends on a detail about the user's own situation that the question does not specify (such as which product they own, their role, their contract type, or their exact travel date), do not guess and do not answer for every possible case. Instead, respond with a direct, specific question addressed to the user asking for that detail (for example: "Which product do you own - the Numi 2.0 or the Verdera Voice?").

If the user's own message already states that detail (for example, they say which role they hold, name the specific product, or give a date), use what they stated together with the context to answer directly for that specific case - do not ask for clarification when the necessary detail has already been given.

If the context does not contain enough information to answer confidently, and no follow-up question could make it sufficient to answer, respond exactly with:

"I don't have enough information to answer confidently"

followed by a brief note on what information is missing, rather than guessing or inferring beyond the context.

When you do answer, be concise. Write in clean prose without bracketed citations, chunk numbers, or other inline reference markers - do not mention "Chunk" or similar labels in your answer.

VERIFY_ANSWER_SYSTEM_PROMPT

You are a quality-control reviewer for an enterprise knowledge assistant. You will be given a draft answer and the source context chunks it was supposed to be based on.

Evaluate whether the draft is fully grounded in the source chunks (no unsupported claims, no facts absent from the context) and decide how confident and actionable the draft is.

Important rule: if the draft asks the user a direct, specific follow-up question about

their own situation (e.g. which product they own, their travel destination, their purchase date) in order to proceed, then action MUST be "clarify" - this is a genuine request for missing user-specific detail, not a refusal. If instead the draft states that it doesn't have enough information, cannot answer confidently, or otherwise contains no substantive answer AND does not ask such a specific follow-up question, then action MUST be "escalate" - regardless of how well-reasoned or appropriate that refusal is. The "answer" action is only for drafts that provide an actual grounded answer to the question, never for drafts that ask for clarification or that correctly decline to answer. A well-written refusal or clarifying question is still not a confident answer.

"grounded" must be true only when action is "answer" and the draft provides a real, evidence-backed response supported by the source chunks. In every other case - "escalate" (whether because the draft refused to answer or because it contained ungrounded content) or "clarify" - "grounded" must be false, since there is no substantive answer being made that could be evaluated as grounded.

Respond with ONLY a single valid JSON object (no markdown formatting, no surrounding text) with exactly these fields:

```
{
  "grounded": <true only if action is "answer" and the draft is a real answer fully
               supported by the source chunks; false in every other case, including
               "escalate" and "clarify">,
  "confidence": <"high", "medium", or "low">,
  "action": <"answer" if the draft provides an actual grounded answer and is ready to
             send as-is, "clarify" if the user's question needs clarification before a
             confident answer is possible, or "escalate" if this should go to a human -
             including whenever the draft itself declines to answer or says it lacks
             enough information>,
  "reasoning": <a brief string explaining the evaluation>
}
```

3. Design Rationale

Each of these decisions came out of a specific failure the eval harness caught, not from guessing upfront what might go wrong. That is deliberate — the prompts above are the result of testing against 20 hand-written questions and fixing what actually broke.

3.1 Why verification is a separate call, not part of drafting

An earlier version asked the drafting model to also self-report its own confidence in the same response. In practice, a model grading its own work tends to rate itself favorably. Splitting verification into a second, independent call — one that only ever sees the draft and the source chunks, never the instruction to "be helpful" that shaped the draft — gives the review a genuinely separate vantage point, closer to how a second reviewer would check a colleague's work rather than the same person re-reading their own answer.

3.2 The escalate-on-refusal rule

The most important bug caught during evaluation: when the draft agent correctly said *"I don't have enough information to answer confidently,"* the verification agent was originally rating that refusal as `grounded: true, confidence: high, action: answer` — because, taken on its own, the refusal text was an accurate, well-supported statement. The system would then have shipped a non-answer to the user labelled as a confident one, which defeats the entire point of having a verification layer. The fix is the explicit rule in the VERIFY prompt: a refusal is never an "answer," no matter how well-reasoned it is, and must always route to `escalate`. This was verified by rerunning the exact test case that exposed it and confirming the action flipped to `escalate`.

3.3 The grounded/action consistency rule

A follow-on issue after the fix above: `grounded: true` could still appear alongside `action: escalate`, which is a confusing state to log or show a reviewer — "grounded" is supposed to mean "safe to trust as-is." The prompt now states directly that `grounded` must be false whenever `action` is not "answer", so the two fields can never contradict each other downstream.

3.4 One rule for missing detail: ask, don't guess and don't enumerate

An earlier version tried to be clever here: when a question depended on a variable the knowledge base fully enumerated (e.g. an approval limit that differs by role), it answered by listing every case instead of asking which one applied - the reasoning being that dumping all the values was more helpful than an extra round trip. In practice this created an inconsistent, hard-to-predict system: two questions that looked the same to a user ("what's my approval limit" vs. "how long will contract review take") were handled differently depending on a subtle judgment call about whether the missing detail was a "document property" or a "personal fact," and that line did not hold up under questioning - the prompt's own wording named role as an example of a variable to enumerate, yet a first-person phrasing of a role question still got asked about rather than answered. Rather than trying to patch that distinction further, the rule was simplified to one sentence: if a detail about the user's own situation is missing and not already stated in their message, ask directly - never guess, and never answer for every possible case. A separate check was added for the regression this caused: if the user's message already states the detail (e.g. "I'm a manager and want to buy..." already gives the role), answer directly using what they stated instead of asking anyway. This trades a small amount of helpfulness on a handful of fully-enumerable questions for a rule that is far easier to state, predict, and defend: the system asks about anything it doesn't already know about the user, full stop.

3.5 Citation relevance threshold

Retrieval returns the top-k chunks for a query, but not every returned chunk is necessarily relevant enough to show the user as a citation — a chunk can rank in the top few purely by keyword overlap without actually supporting the answer. A similarity-score cutoff (raised from 0.38 to 0.40 after spot-checking citation quality) is applied only to what's shown to the user as a source citation. It is deliberately not applied to what's fed into drafting and verification, because multi-hop questions sometimes need a chunk that scores lower on raw similarity but is still necessary to combine with a higher-scoring one for a complete answer.

3.6 Model consolidation onto Flash-Lite

The original design used a stronger model for the drafting step and Flash-Lite for the cheaper classify/verify steps. That plan met a hard constraint almost immediately: the stronger tier's free daily quota is 20 requests a day, which a single 20-question eval run burns through by itself. Flash-Lite's quota is far more generous, so drafting was moved onto it as well — all three calls now share one model. This is a real engineering decision made after hitting the constraint in practice, not a compromise hidden after the fact, and the architecture would not need to change to move back to a stronger model on a paid plan.

3.7 Client-side rate limiting

Consolidating onto one model then exposed its per-minute limit (15 requests/minute on the free tier): a single query can trigger up to three calls internally (classify, draft, verify), so a short burst of user activity could exceed the limit even with the daily quota untouched. A small client-side limiter in `agent.s.py` tracks the timestamp of the last Gemini call and sleeps as needed to keep calls at least ~4.2 seconds apart, regardless of which function triggers them. This is also why the deployed app takes 10-15 seconds to respond — it is intentional pacing, not a performance problem.

That interval was later found to be too tight: at exactly 4.2 seconds, a rolling 60-second window sits precisely at the 15-request ceiling, so ordinary latency jitter was enough to tip it over and crash a full evaluation run partway through. The interval was widened to 5.0 seconds (12 requests/minute), giving genuine headroom instead of running the quota at its exact boundary.

3.8 Distinguishing a genuine clarifying question from a refusal

Caught not by the automated eval set but by manually testing the deployed app: three questions that should have asked the user for a missing detail about their own situation (their approval role, their trip's per diem tier, which product they owned) were instead either escalated or answered outright. The root cause was that `DRAFT_ANSWER_SYSTEM_PROMPT` had only one fallback template - the same "I don't have enough information to answer confidently" wording - for both a genuinely unanswerable question and a question that just needed one more detail from the user, while `VERIFY_ANSWER_SYSTEM_PROMPT`'s escalate-on-refusal rule (Section 3.2) then routed every instance of that template to `escalate`, regardless of which kind it was. The fix gives the draft a distinct phrasing instruction for a genuine clarifying question, and updates the verify rule to check whether the draft asks a direct follow-up question (route to `clarify`) versus states a lack of information with no follow-up (route to `escalate`). Verified by rerunning the exact three live-tested questions and confirming all three now return `clarify`, then by a full eval re-run showing no regressions.

4. Development Workflow

The system was built iteratively inside VS Code: defining the architecture and scope up front, writing the knowledge-base content and prompts directly, and then building out the source files (retrieval pipeline, agent calls, Streamlit UI, evaluation harness) against those specifications. Every fix described in Section 3 followed the same loop:

- Run the 20-question evaluation harness (`eval/run_eval.py`) and look at exactly which cases disagreed with the expected outcome, not just the pass rate.
- Read the actual reasoning field the verification agent produced for the failing case, to see *why* it made that call rather than guessing.
- Change one prompt rule at a time to address that specific failure, rerun only the affected test case first to confirm the fix, then rerun the full set to check nothing else regressed.

Two examples worth calling out specifically, since they were the clearest instances of a prompt bug rather than a genuinely ambiguous case: the refusal-marked-as-answer bug (Section 3.2), caught on a question about stock option vesting that isn't covered anywhere in the knowledge base by design — the draft correctly said it didn't know, but verification was initially about to ship that as a confident answer; and the missing-detail regression (Section 3.4), caught when simplifying the clarification rule to a single sentence broke a question that had already given the needed detail — the draft asked the user for their role even though the question had already stated it.

The project also went through a budget-driven architecture pivot partway through: it was originally built against the Anthropic API, but that requires a funded billing account with no meaningful free tier, which wasn't an option here. The entire backend was moved to Google's Gemini API instead (genuinely free, no card required), which took a rewrite of the agent-calling code but no change to the pipeline architecture itself, since the interface between stages (retrieve → draft → verify → route) never depended on which model was making the call.

The finished system is deployed on Streamlit Community Cloud so it can be evaluated without any local setup: `kohler-enterprise-agent.streamlit.app`.

5. Evaluation Summary

`eval/test_set.json` contains 20 hand-written test questions spanning four categories: straightforward questions answerable from a single document, multi-hop questions that require combining two documents, genuinely ambiguous questions, and questions on topics deliberately left out of the knowledge base entirely (to test that the system escalates rather than fabricates an answer).

Metric	Result
Test cases	20, spanning answer / clarify / escalate outcomes
Current status	Every case routes to its expected action; see Sections 3.2, 3.4, and 3.8 for the specific fixes involved
Hallucinated or fabricated facts observed	0, across every run performed

Test-set label corrections applied	None in the current version - an earlier attempt at an enumerate-instead-of-ask rule (Section 3.4) briefly justified relabeling one case, but was reverted along with the rule itself once it proved inconsistent
------------------------------------	---

Every fix in this document was reached the same way: running the eval, reading exactly which case disagreed and why, and fixing the specific prompt rule that caused it - never by loosening a rule just to make a stubborn case pass. That includes reversing a rule that turned out to be a mistake: an attempt at enumerating fully-covered variables instead of asking about them (Section 3.4) was tried, found to produce inconsistent behavior on similar-looking questions, and replaced with a single, simpler clarification rule instead - changing the code back to match the original test expectations, rather than changing the expectations to match the code. A hand-written 20-case set is small enough that a clean pass rate is not, by itself, strong evidence of anything - what matters more for an enterprise assistant is the zero-hallucination result: in every evaluation run performed during development, the system never stated a fact that was not present in its retrieved source documents. When it did not know something, it said so, consistently, rather than guessing.

Known limitations

- Response latency is 10-15 seconds per answer, a direct consequence of the client-side rate limiting described in Section 3.7 rather than a modeling limitation.
- The answer / clarify / escalate decision is not perfectly consistent on genuinely ambiguous questions — asking the same borderline question twice can produce two different but equally reasonable outcomes. This reflects the underlying uncertainty in those specific questions, not an implementation defect, and is not something that can be fully eliminated without removing the ambiguity from the question itself.